

Inżynieria danych

Wykład 9: transakcje (cz. I)

Katarzyna Grygiel

Semestr letni 2025/2026

Dostęp do kont bankowych

Kilku klientów chce mieć dostęp do swoich rachunków w tym samym czasie, przynajmniej jeden zmienia saldo konta poprzez wypłatę, wpłatę lub przelew.

Co możemy zrobić?

Dostęp do kont bankowych

Kilku klientów chce mieć dostęp do swoich rachunków w tym samym czasie, przynajmniej jeden zmienia saldo konta poprzez wypłatę, wpłatę lub przelew.

Co możemy zrobić?

- W danym momencie pozwolić na dokonanie zmiany tylko jednemu klientowi:
 - nie będzie konfliktów,
 - dostęp do kont będzie zablokowany dla pozostałych klientów,
 - utworzy się kolejka zapytań dla prawie wszystkich klientów.

Dostęp do kont bankowych

Kilku klientów chce mieć dostęp do swoich rachunków w tym samym czasie, przynajmniej jeden zmienia saldo konta poprzez wypłatę, wpłatę lub przelew.

Co możemy zrobić?

- W danym momencie pozwolić na dokonanie zmiany tylko jednemu klientowi:
 - nie będzie konfliktów,
 - dostęp do kont będzie zablokowany dla pozostałych klientów,
 - utworzy się kolejka zapytań dla prawie wszystkich klientów.
- W danym momencie pozwolić na dokonanie zmian klientom, którzy operują na niezależnych kontach:
 - nie będzie konfliktów,
 - dostęp do kont będzie zablokowany dla klientów, którzy chcą operować na „zajętych” krotkach,
 - trzeba ustalić kolejność dostępu do konta,
 - utworzy się kolejka zapytań dla poszczególnych kont.

Przelewy

Mamy na koncie 100 zł, które chcemy przelać równocześnie dwóm osobom. Jak zagwarantować, że uda się wykonać maksymalnie jeden przelew?

Przelewy

Mamy na koncie 100 zł, które chcemy przelać równocześnie dwóm osobom. Jak zagwarantować, że uda się wykonać maksymalnie jeden przelew?

Dział finansowy dużej firmy przelewa pensje wszystkim pracownikom w tym samym momencie. Jak zagwarantować, że wszystkie operacje przebiegną szybko i poprawnie?

Przelewy

Mamy na koncie 100 zł, które chcemy przelać równocześnie dwóm osobom. Jak zagwarantować, że uda się wykonać maksymalnie jeden przelew?

Dział finansowy dużej firmy przelewa pensje wszystkim pracownikom w tym samym momencie. Jak zagwarantować, że wszystkie operacje przebiegną szybko i poprawnie?

Dowolne przeploty wielu transakcji mogą prowadzić do niespójności. Jeśli jest ona chwilowa, to możemy sobie na nią nieraz pozwolić. Nie możemy jednak dopuścić do trwałego rozspójnienia danych.

Aby w jakiś sposób rozwiązać powyższe problemy, musimy formalnie zdefiniować kryteria poprawności współbieżnego wykonywania kilku transakcji.

Czytamy i piszemy

Operacja **odczytu** pobiera z relacji wartość krotki (lub ewentualnie innego obiektu bazodanowego). Taką operację dla krotki A będziemy oznaczali przez $r(A)$.

Czytamy i piszemy

Operacja **odczytu** pobiera z relacji wartość krotki (lub ewentualnie innego obiektu bazodanowego). Taką operację dla krotki A będziemy oznaczali przez $r(A)$.

Operacja **zapisu** zmienia w relacji wartość krotki (lub ewentualnie innego obiektu bazodanowego). Wyróżniamy trzy rodzaje operacji zapisu:

- INSERT, czyli wstawianie nowej krotki,
- UPDATE, czyli zmianę wartości jednego lub kilku atrybutów w danej krotce,
- DELETE, czyli usuwanie krotki z relacji.

Operację zapisu dla krotki A będziemy oznaczać przez $w(A)$.

Transakcja: wszystko albo nic

Transakcja składa się z jednej lub wielu operacji czytania i pisania. Wraz z operacjami wchodzącymi w skład transakcji ustalony jest porządek ich wykonywania. Ciąg wszystkich operacji dla danej transakcji traktujemy jako całość, czyli albo wykonane zostaną wszystkie operacje, albo żadna z nich.

Transakcja: wszystko albo nic

Transakcja składa się z jednej lub wielu operacji czytania i pisania. Wraz z operacjami wchodzącymi w skład transakcji ustalony jest porządek ich wykonywania. Ciąg wszystkich operacji dla danej transakcji traktujemy jako całość, czyli albo wykonane zostaną wszystkie operacje, albo żadna z nich.

Początek każdej transakcji oznaczać będziemy przez `b` (`begin`), zatwierdzenie transakcji przez `c` (`commit`), natomiast jej wycofanie przez `a` (`abort`, również `rollback`).

Relację poprzedzania będziemy oznaczać strzałką: zapis $x \rightarrow y$ oznacza, że operacja x jest wykonywana przed operacją y .

Przykład i ograniczone informacje

Przykład. Rozważmy transakcję T_1 opisującą przelew z konta nr 2 na konto nr 9. Odpowiednie krotki dla tych kont oznaczmy przez K_2 i K_9 . Wtedy transakcję T_1 opiszemy ciągiem

$$b_1 \rightarrow r_1(K_2) \rightarrow w_1(K_2) \rightarrow r_1(K_9) \rightarrow w_1(K_9) \rightarrow c_1.$$

Zauważmy, że nigdzie nie zapisujemy, jakie konkretnie zmiany dotyczą wartości krotek K_2 oraz K_9 .

Przykład i ograniczone informacje

Przykład. Rozważmy transakcję T_1 opisującą przelew z konta nr 2 na konto nr 9. Odpowiednie krotki dla tych kont oznaczmy przez K_2 i K_9 . Wtedy transakcję T_1 opiszemy ciągiem

$$b_1 \rightarrow r_1(K_2) \rightarrow w_1(K_2) \rightarrow r_1(K_9) \rightarrow w_1(K_9) \rightarrow c_1.$$

Zauważmy, że nigdzie nie zapisujemy, jakie konkretnie zmiany dotyczą wartości krotek K_2 oraz K_9 .

Dzieje się tak dlatego, że często zmiany dokonywane są z poziomu aplikacji, w związku z czym SZBD nie wie, jakie wartości się pojawiają. System jest natomiast w stanie określić, jakiego typu operacje (odczytu czy zapisu) są dokonywane i których krotek (obiektów bazodanowych) dotyczą potencjalne zmiany.

W związku z tym system tylko na podstawie takich informacji powinien zdecydować, czy określone operacje odczytu i zapisu są dozwolone, czy też nie.

ACID

Współbieżne wykonywanie transakcji wymaga zachowania własności **ACID**:

- **atomowość** (*atomicity*): transakcja nie może być wykonana częściowo, czyli albo wykonane zostaną wszystkie operacje wchodzące w jej skład, albo żadna;

ACID

Współbieżne wykonywanie transakcji wymaga zachowania własności **ACID**:

- **atomowość** (*atomicity*): transakcja nie może być wykonana częściowo, czyli albo wykonane zostaną wszystkie operacje wchodzące w jej skład, albo żadna;
- **integralność** (*consistency*): po zatwierdzeniu transakcji muszą być spełnione wszystkie warunki poprawności nałożone na bazę danych;

ACID

Współbieżne wykonywanie transakcji wymaga zachowania własności **ACID**:

- **atomowość** (*atomicity*): transakcja nie może być wykonana częściowo, czyli albo wykonane zostaną wszystkie operacje wchodzące w jej skład, albo żadna;
- **integralność** (*consistency*): po zatwierdzeniu transakcji muszą być spełnione wszystkie warunki poprawności nałożone na bazę danych;
- **izolacja** (*isolation*): efekt równoległego wykonania dwóch lub więcej transakcji musi być szeregowalny;

ACID

Współbieżne wykonywanie transakcji wymaga zachowania własności **ACID**:

- **atomowość** (*atomicity*): transakcja nie może być wykonana częściowo, czyli albo wykonane zostaną wszystkie operacje wchodzące w jej skład, albo żadna;
- **integralność** (*consistency*): po zatwierdzeniu transakcji muszą być spełnione wszystkie warunki poprawności nałożone na bazę danych;
- **izolacja** (*isolation*): efekt równoległego wykonania dwóch lub więcej transakcji musi być szeregowalny;
- **trwałość** (*durability*): po udanym zakończeniu transakcji jej efekty na stałe pozostają w bazie danych.

Atomowość

Każda transakcja jest niepodzielna, nie można wykonać tylko kilku operacji. Albo zatwierdzone są wszystkie operacje, albo wszystkie są wycofane.

Atomowość

Każda transakcja jest niepodzielna, nie można wykonać tylko kilku operacji. Albo zatwierdzone są wszystkie operacje, albo wszystkie są wycofane.

Transakcja może być wycofana przez użytkownika aplikacji lub samą aplikację, jak również przez system bazodanowy (na przykład gdy operacje prowadzą do rozpójdzenia danych lub doszło do zakleszczenia).

Atomowość

Każda transakcja jest niepodzielna, nie można wykonać tylko kilku operacji. Albo zatwierdzone są wszystkie operacje, albo wszystkie są wycofane.

Transakcja może być wycofana przez użytkownika aplikacji lub samą aplikację, jak również przez system bazodanowy (na przykład gdy operacje prowadzą do rozspójnienia danych lub doszło do zakleszczenia).

Przykład. Pełna transakcja, która zostaje zatwierdzona:

$$b_1 \rightarrow r_1(K_2) \rightarrow w_1(K_2) \rightarrow r_1(K_9) \rightarrow w_1(K_9) \rightarrow c_1.$$

Transakcja, która zostaje przerwana, a wszystkie zmiany (w tym wypadku operacja $w_1(K_2)$) wycofane:

$$b_1 \rightarrow r_1(K_2) \rightarrow w_1(K_2) \rightarrow r_1(K_9) \rightarrow a_1.$$

Atomowość

Baza danych musi być zatem chroniona przed zmianami, które wynikają z wykonania tylko części operacji należących do transakcji.

Co więcej, dokonywane zmiany, które jeszcze nie zostały zatwierdzone, nie powinny być widoczne dla innych transakcji, czyli nie są dla nich bezpośrednio widoczne operacje zapisu przesyłane do bazy danych.

Atomowość

Baza danych musi być zatem chroniona przed zmianami, które wynikają z wykonania tylko części operacji należących do transakcji.

Co więcej, dokonywane zmiany, które jeszcze nie zostały zatwierdzone, nie powinny być widoczne dla innych transakcji, czyli nie są dla nich bezpośrednio widoczne operacje zapisu przesyłane do bazy danych.

Aby zapewnić atomowość, systemy bazodanowe wykorzystują jedną z metod:

- logowanie zmian (często stosowane),
- *shadow paging*, czyli tworzenie kopii stron plików, na których dokonywane są zmiany i dopiero po zatwierdzeniu transakcji umożliwienie ich odczytu (rzadko stosowane).

Spójność

Baza danych powinna w każdym momencie poprawnie modelować jakąś część rzeczywistości, dlatego po zatwierdzeniu transakcji nie powinny być naruszone żadne z nałożonych na bazę warunków spójności.

W pewnych przypadkach, w zależności od ustawień, pojedyncze operacje wchodzące w skład transakcji mogą jednak chwilowo naruszać istniejące ograniczenia.

Spójność

Baza danych powinna w każdym momencie poprawnie modelować jakąś część rzeczywistości, dlatego po zatwierdzeniu transakcji nie powinny być naruszone żadne z nałożonych na bazę warunków spójności.

W pewnych przypadkach, w zależności od ustawień, pojedyncze operacje wchodzące w skład transakcji mogą jednak chwilowo naruszać istniejące ograniczenia.

Przykład. Dla przelewów wewnętrznych danego banku wynik poniższego zapytania przed i po zatwierdzeniu transakcji powinien być taki sam:

```
SELECT SUM(stan_konta) FROM konta;
```


Izolacja

Izolacja oznacza, że dwie transakcje wykonywane w tym samym czasie nie wpływają na siebie.

Z punktu widzenia pojedynczej transakcji jest ona jedyną transakcją, która jest właśnie wykonywana. To, w jaki sposób pozostałe transakcje wpływają na stan bazy danych, nie jest dla niej widoczne.

Izolacja

Izolacja oznacza, że dwie transakcje wykonywane w tym samym czasie nie wpływają na siebie.

Z punktu widzenia pojedynczej transakcji jest ona jedyną transakcją, która jest właśnie wykonywana. To, w jaki sposób pozostałe transakcje wpływają na stan bazy danych, nie jest dla niej widoczne.

Aby warunek izolacji był spełniony, systemy wykorzystują różne protokoły, które można podzielić na dwie kategorie:

- pesymistyczne, które nie pozwalają na zaistnienie problemu,
- optymistyczne, które zakładają, że problemy pojawiają się rzadko i dopiero po ich pojawieniu wdrażają rozwiązania.

Izolacja

Rozważmy dwa konta, na których znajduje się 1000 zł. Jakie mogą być wyniki wykonania poniższych transakcji?

T_1	T_2
begin	begin
$A := A - 100$	$A := 1,1 \cdot A$
$B := B + 100$	$B := 1,1 \cdot B$
commit	commit

W zależności od kolejności wykonywania poszczególnych operacji, ostateczne wyniki mogą się różnić:

$$A = 1000, B = 1200 \quad \text{lub} \quad A = 990, B = 1210.$$

Niezależnie jednak od tego, w jaki sposób przeprowadzone zostaną transakcje, suma stanu obu kont powinna być stała:

$$A + B = 1,1 \cdot 2000 = 2200.$$

Trwałość

Trwałość oznacza, że dla każdej zatwierdzonej transakcji wszystkie zmiany przez nią dokonane są stałe i nawet w przypadku twardych awarii nie zostaną utracone.

Trwałość

Trwałość oznacza, że dla każdej zatwierdzonej transakcji wszystkie zmiany przez nią dokonane są stałe i nawet w przypadku twardych awarii nie zostaną utracone.

Oznacza to, że baza danych musi być chroniona przed wszelkiego rodzaju awariami (np. utrata zasilania) i błędami (np. w oprogramowaniu). Zakładamy dla uproszczenia, że awaria może spowodować utratę danych z pamięci głównej, ale nigdy dane zapisane na dysku nie są tracone.

Trwałość

Trwałość oznacza, że dla każdej zatwierdzonej transakcji wszystkie zmiany przez nią dokonane są stałe i nawet w przypadku twardych awarii nie zostaną utracone.

Oznacza to, że baza danych musi być chroniona przed wszelkiego rodzaju awariami (np. utrata zasilania) i błędami (np. w oprogramowaniu). Zakładamy dla uproszczenia, że awaria może spowodować utratę danych z pamięci głównej, ale nigdy dane zapisane na dysku nie są tracone.

Trwałość można zapewnić następująco:

- zapisując zmiany przeprowadzane przez transakcje na dysku przez zakończeniem transakcji,
- przechowując wystarczające informacje o zmianach, dzięki czemu możliwe jest odtworzenie stanu bazy sprzed awarii.

Realizacja transakcji

Niech $\mathcal{T} = \{T_1, \dots, T_n\}$ będzie zbiorem transakcji. **Realizacją** (lub **planem wykonania**) transakcji ze zbioru $\mathcal{T}$ nazywamy częściowy porządek $(\mathcal{O}_{\mathcal{T}}, \leq)$, gdzie:

- $\mathcal{O}_{\mathcal{T}}$ jest zbiorem wszystkich operacji należących do transakcji $T_1, \dots, T_n$ wraz z początkami i zatwierdzeniami/wycofaniami każdej z transakcji,

Realizacja transakcji

Niech $\mathcal{T} = \{T_1, \dots, T_n\}$ będzie zbiorem transakcji. **Realizacją** (lub **planem wykonania**) transakcji ze zbioru $\mathcal{T}$ nazywamy częściowy porządek $(\mathcal{O}_{\mathcal{T}}, \leq)$, gdzie:

- $\mathcal{O}_{\mathcal{T}}$ jest zbiorem wszystkich operacji należących do transakcji $T_1, \dots, T_n$ wraz z początkami i zatwierdzeniami/wycofaniami każdej z transakcji,
- porządek $\leq$ jest zgodny z każdym z porządków $\rightarrow_i$ dla $i = 1, \dots, n$, gdzie $\rightarrow_i$ określa kolejność wykonywania operacji dla i -tej transakcji,

Realizacja transakcji

Niech $\mathcal{T} = \{T_1, \dots, T_n\}$ będzie zbiorem transakcji. **Realizacją** (lub **planem wykonania**) transakcji ze zbioru $\mathcal{T}$ nazywamy częściowy porządek $(\mathcal{O}_{\mathcal{T}}, \leq)$, gdzie:

- $\mathcal{O}_{\mathcal{T}}$ jest zbiorem wszystkich operacji należących do transakcji $T_1, \dots, T_n$ wraz z początkami i zatwierdzeniami/wycofaniami każdej z transakcji,
- porządek $\leq$ jest zgodny z każdym z porządków $\rightarrow_i$ dla $i = 1, \dots, n$, gdzie $\rightarrow_i$ określa kolejność wykonywania operacji dla i -tej transakcji,
- jeżeli dwie operacje o i o' żądają dostępu do tej samej krotki (lub tego samego obiektu bazodanowego) i co najmniej jedna z nich jest operacją zapisu, to zachodzi $o \leq o'$ lub $o' \leq o$.

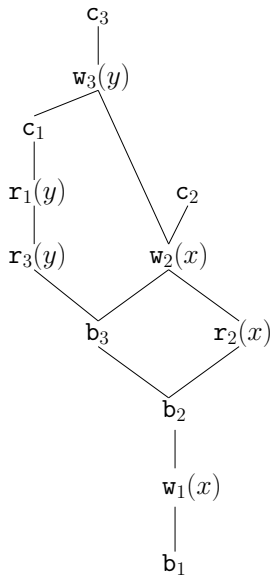
Realizacja transakcji

Niech $\mathcal{T} = \{T_1, \dots, T_n\}$ będzie zbiorem transakcji. **Realizacją** (lub **planem wykonania**) transakcji ze zbioru $\mathcal{T}$ nazywamy częściowy porządek $(\mathcal{O}_{\mathcal{T}}, \leq)$, gdzie:

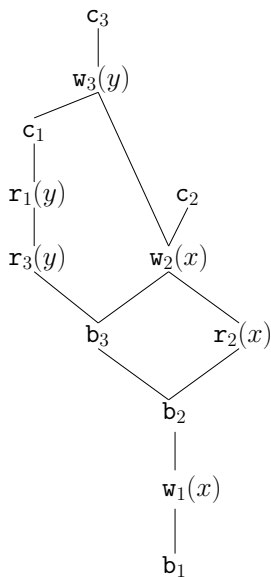
- $\mathcal{O}_{\mathcal{T}}$ jest zbiorem wszystkich operacji należących do transakcji $T_1, \dots, T_n$ wraz z początkami i zatwierdzeniami/wycofaniami każdej z transakcji,
- porządek $\leq$ jest zgodny z każdym z porządków $\rightarrow_i$ dla $i = 1, \dots, n$, gdzie $\rightarrow_i$ określa kolejność wykonywania operacji dla i -tej transakcji,
- jeżeli dwie operacje o i o' żądają dostępu do tej samej krotki (lub tego samego obiektu bazodanowego) i co najmniej jedna z nich jest operacją zapisu, to zachodzi $o \leq o'$ lub $o' \leq o$.

Jeżeli porządek $\leq$ jest liniowy, to będziemy również używać oznaczenia $\rightarrow$.

Transakcje sekwencyjne i współbieżne

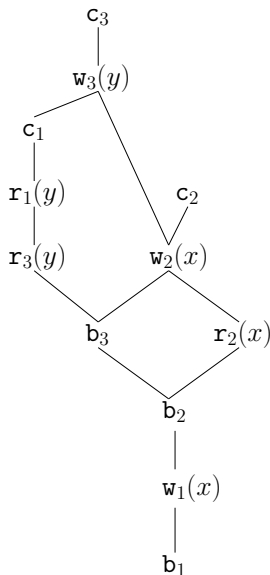


Transakcje sekwencyjne i współbieżne



Realizację nazywamy **sekwencyjną**, jeżeli dla każdych dwóch transakcji wszystkie operacje jednej z nich są poprzedzone wszystkimi operacjami drugiej.

Transakcje sekwencyjne i współbieżne



Realizację nazywamy **sekwencyjną**, jeżeli dla każdych dwóch transakcji wszystkie operacje jednej z nich są poprzedzone wszystkimi operacjami drugiej.

W przeciwnym wypadku mówimy o realizacji **współbieżnej**.

Realizacje poprawne

Rozważmy zbiór transakcji $\mathcal{T} = \{T_1, \dots, T_n\}$.

Dwie realizacje $\mathcal{T}$ nazwiemy **równoważnymi**, jeżeli dają ten sam stan bazy danych oraz zbiór wartości odczytywanych przez każdą z transakcji T_i jest taki sam.

Realizacje poprawne

Rozważmy zbiór transakcji $\mathcal{T} = \{T_1, \dots, T_n\}$.

Dwie realizacje $\mathcal{T}$ nazwiemy **równoważnymi**, jeżeli dają ten sam stan bazy danych oraz zbiór wartości odczytywanych przez każdą z transakcji T_i jest taki sam.

Realizację zbioru transakcji $\mathcal{T}$ nazwiemy **szeregowalną** lub **poprawną**, jeżeli jest ona równoważna pewnej sekwencyjnej realizacji $\mathcal{T}$.

Realizacje poprawne

Rozważmy zbiór transakcji $\mathcal{T} = \{T_1, \dots, T_n\}$.

Dwie realizacje $\mathcal{T}$ nazwiemy **równoważnymi**, jeżeli dają ten sam stan bazy danych oraz zbiór wartości odczytywanych przez każdą z transakcji T_i jest taki sam.

Realizację zbioru transakcji $\mathcal{T}$ nazwiemy **szeregowalną** lub **poprawną**, jeżeli jest ona równoważna pewnej sekwencyjnej realizacji $\mathcal{T}$.

Zauważmy, że zgodnie z definicją każda realizacja sekwencyjna jest poprawna, czyli dla dowolnych dwóch transakcji T_i i T_j wykonanie najpierw T_i , a następnie T_j oraz wykonanie najpierw T_j , a następnie T_i nie narusza poprawności.

Oznacza to, że dwie poprawne realizacje mogą prowadzić do różnych stanów bazy. Ponadto wartości danych odczytywanych przez określoną transakcję w dwóch różnych realizacjach również mogą być różne.

Poprawne realizacje transakcji

plan 1

T_1	T_2
begin	
$A := A - 100$	
$B := B + 100$	
commit	
	begin
	$A := 1,1A$
	$B := 1,1B$
	commit

bilans: $A = 990, B = 1210$

plan 2

T_1	T_2
	begin
	$A := 1,1A$
	$B := 1,1B$
	commit
begin	
$A := A - 100$	
$B := B + 100$	
commit	

bilans: $A = 1000, B = 1200$

Poprawne realizacje transakcji

plan 1

T_1	T_2
begin	
$A := A - 100$	
$B := B + 100$	
commit	
	begin
	$A := 1,1A$
	$B := 1,1B$
	commit

bilans: $A = 990, B = 1210$

plan 2

T_1	T_2
	begin
	$A := 1,1A$
	$B := 1,1B$
	commit
begin	
$A := A - 100$	
$B := B + 100$	
commit	

bilans: $A = 1000, B = 1200$

Powyższe dwa plany wykonania transakcji nie są sobie równoważne, ponieważ prowadzą do różnych stanów bazy.

Obie realizacje są jednak poprawne, ponieważ są sekwencyjne. Z punktu widzenia SZBD nie ma znaczenia fakt, że ostateczny stan obu kont jest różny dla każdego z planów.

Niepoprawna realizacja transakcji

plan 3

T_1	T_2
begin	
$A := A - 100$	
	begin
	$A := 1,1A$
	$B := 1,1B$
	commit
$B := B + 100$	
commit	

bilans: $A = 990$, $B = 1200$

Powyższy plan jest niepoprawny, ponieważ ostateczny wynik nie jest równoważny wynikowi żadnej z sekwencyjnych realizacji transakcji T_1 i T_2 .

Jeszcze jedna poprawna realizacja transakcji

plan 4

T_1	T_2
begin	
$A := A - 100$	
	begin
	$A := 1,1A$
$B := B + 100$	
commit	
	$B := 1,1B$
	commit

bilans: $A = 990$, $B = 1210$

Powyższy plan jest poprawny, ponieważ jest równoważnemu sekwencyjnemu planowi pierwszemu (najpierw wykonanie transakcji T_1 , a następnie T_2).

Problem szeregowalności

Rozważmy dwie realizacje (pomijamy b_1 i b_2):

$$r_2(x) \rightarrow w_2(x) \rightarrow r_1(x) \rightarrow r_1(y) \rightarrow w_2(y) \rightarrow c_1 \rightarrow c_2$$

$$r_1(x) \rightarrow r_1(y) \rightarrow r_2(y) \rightarrow c_1 \rightarrow r_2(x) \rightarrow w_2(x) \rightarrow w_2(y) \rightarrow c_2$$

Która z nich jest poprawna?

Problem szeregowalności

Rozważmy dwie realizacje (pomijamy b_1 i b_2):

$$r_2(x) \rightarrow w_2(x) \rightarrow r_1(x) \rightarrow r_1(y) \rightarrow w_2(y) \rightarrow c_1 \rightarrow c_2$$

$$r_1(x) \rightarrow r_1(y) \rightarrow r_2(y) \rightarrow c_1 \rightarrow r_2(x) \rightarrow w_2(x) \rightarrow w_2(y) \rightarrow c_2$$

Która z nich jest poprawna?

PROBLEM SZEREGOWALNOŚCI

INPUT: Realizacja R dla zbioru transakcji $\mathcal{T} = \{T_1, \dots, T_n\}$

OUTPUT: Czy R jest szeregowalna?

Problem szeregowalności

Rozważmy dwie realizacje (pomijamy b_1 i b_2):

$$r_2(x) \rightarrow w_2(x) \rightarrow r_1(x) \rightarrow r_1(y) \rightarrow w_2(y) \rightarrow c_1 \rightarrow c_2$$

$$r_1(x) \rightarrow r_1(y) \rightarrow r_2(y) \rightarrow c_1 \rightarrow r_2(x) \rightarrow w_2(x) \rightarrow w_2(y) \rightarrow c_2$$

Która z nich jest poprawna?

PROBLEM SZEREGOWALNOŚCI

INPUT: Realizacja R dla zbioru transakcji $\mathcal{T} = \{T_1, \dots, T_n\}$

OUTPUT: Czy R jest szeregowalna?

Twierdzenie (Papadimitriou, 1979)

Problem szeregowalności jest NP-zupełny.

Operacje konfliktowe

Dwie operacje należące do planu wykonania zapytań nazwiemy **konfliktowymi**, jeśli spełnione są wszystkie trzy warunki:

- operacje należą do dwóch różnych transakcji,
- obie operacje dotyczą tego samego obiektu bazodanowego,
- co najmniej jedna z tych operacji jest operacją zapisu.

Operacje konfliktowe

Dwie operacje należące do planu wykonania zapytań nazwiemy **konfliktowymi**, jeśli spełnione są wszystkie trzy warunki:

- operacje należą do dwóch różnych transakcji,
- obie operacje dotyczą tego samego obiektu bazodanowego,
- co najmniej jedna z tych operacji jest operacją zapisu.

Przykład.

- $r_1(A)$ oraz $r_2(A)$
- $r_1(A)$ oraz $w_1(A)$
- $r_1(A)$ oraz $w_2(A)$
- $w_1(A)$ oraz $w_2(A)$
- $w_1(A)$ oraz $w_2(B)$

Operacje konfliktowe

Dwie operacje należące do planu wykonania zapytań nazwiemy **konfliktowymi**, jeśli spełnione są wszystkie trzy warunki:

- operacje należą do dwóch różnych transakcji,
- obie operacje dotyczą tego samego obiektu bazodanowego,
- co najmniej jedna z tych operacji jest operacją zapisu.

Przykład.

- | | |
|--------------------------|----------------|
| • $r_1(A)$ oraz $r_2(A)$ | bezkonfliktowa |
| • $r_1(A)$ oraz $w_1(A)$ | bezkonfliktowa |
| • $r_1(A)$ oraz $w_2(A)$ | konfliktowa |
| • $w_1(A)$ oraz $w_2(A)$ | konfliktowa |
| • $w_1(A)$ oraz $w_2(B)$ | bezkonfliktowa |

Anomalie wynikające z konfliktów

- Niepowtarzalne odczyty (konflikt read-write, transakcja odczytuje różne wartości):

$$r_1(A) \rightarrow r_2(A) \rightarrow w_2(A) \rightarrow c_2 \rightarrow r_1(A) \rightarrow c_1$$

Anomalie wynikające z konfliktów

- **Niepowtarzalne odczyty** (konflikt read-write, transakcja odczytuje różne wartości):

$$r_1(A) \rightarrow r_2(A) \rightarrow w_2(A) \rightarrow c_2 \rightarrow r_1(A) \rightarrow c_1$$

- **Brudne odczyty** (konflikt write-read, transakcja odczytuje zmiany dokonane przez inną, które nie została zatwierdzona):

$$r_1(A) \rightarrow w_1(A) \rightarrow r_2(A) \rightarrow w_2(A) \rightarrow c_2 \rightarrow a_1$$

Anomalie wynikające z konfliktów

- **Niepowtarzalne odczyty** (konflikt read-write, transakcja odczytuje różne wartości):

$$r_1(A) \rightarrow r_2(A) \rightarrow w_2(A) \rightarrow c_2 \rightarrow r_1(A) \rightarrow c_1$$

- **Brudne odczyty** (konflikt write-read, transakcja odczytuje zmiany dokonane przez inną, które nie została zatwierdzona):

$$r_1(A) \rightarrow w_1(A) \rightarrow r_2(A) \rightarrow w_2(A) \rightarrow c_2 \rightarrow a_1$$

- **Nadpisywanie niezatwierdzonych zmian** (konflikt write-write, transakcja nadpisuje dane, które zostały zmienione przez inną niezatwierdzoną transakcję):

$$w_1(A) \rightarrow w_2(A) \rightarrow w_2(B) \rightarrow w_1(B) \rightarrow c_2 \rightarrow c_1$$

Konfliktowa równoważność

Dwie realizacje R_1 i R_2 są **konfliktowo równoważne**, jeżeli każde dwie operacje konfliktowe występują w obu realizacjach w tej samej kolejności.

Konfliktowa równoważność

Dwie realizacje R_1 i R_2 są **konfliktowo równoważne**, jeżeli każde dwie operacje konfliktowe występują w obu realizacjach w tej samej kolejności.

Przykład. Rozważmy następujące realizacje:

$$R_1 : \quad r_1(A) \rightarrow r_2(B) \rightarrow w_1(A) \rightarrow r_2(A) \rightarrow w_2(B) \rightarrow c_2 \rightarrow c_1$$

$$R_2 : \quad r_1(A) \rightarrow w_1(A) \rightarrow r_2(B) \rightarrow r_2(A) \rightarrow w_2(B) \rightarrow c_2 \rightarrow c_1$$

$$R_3 : \quad r_1(A) \rightarrow r_2(B) \rightarrow r_2(A) \rightarrow w_1(A) \rightarrow w_2(B) \rightarrow c_2 \rightarrow c_1$$

Konfliktowa równoważność

Dwie realizacje R_1 i R_2 są **konfliktowo równoważne**, jeżeli każde dwie operacje konfliktowe występują w obu realizacjach w tej samej kolejności.

Przykład. Rozważmy następujące realizacje:

$$R_1 : \quad r_1(A) \rightarrow r_2(B) \rightarrow w_1(A) \rightarrow r_2(A) \rightarrow w_2(B) \rightarrow c_2 \rightarrow c_1$$

$$R_2 : \quad r_1(A) \rightarrow w_1(A) \rightarrow r_2(B) \rightarrow r_2(A) \rightarrow w_2(B) \rightarrow c_2 \rightarrow c_1$$

$$R_3 : \quad r_1(A) \rightarrow r_2(B) \rightarrow r_2(A) \rightarrow w_1(A) \rightarrow w_2(B) \rightarrow c_2 \rightarrow c_1$$

Realizacje R_1 i R_2 są konfliktowo równoważne, natomiast R_3 nie jest konfliktowo równoważna żadnej z nich.

Konfliktowa szeregowalność

Powiemy, że realizacja jest **konfliktowo szeregowalna**, jeżeli jest konfliktowo równoważna pewnej realizacji sekwencyjnej.

Jeżeli możemy otrzymać pewną realizację R z realizacji sekwencyjnej przez zamianę operacji niekonfliktowych, to R jest konfliktowo szeregowalna.

Konfliktowa szeregowalność

Powiemy, że realizacja jest **konfliktowo szeregowalna**, jeżeli jest konfliktowo równoważna pewnej realizacji sekwencyjnej.

Jeżeli możemy otrzymać pewną realizację R z realizacji sekwencyjnej przez zamianę operacji niekonfliktowych, to R jest konfliktowo szeregowalna.

Przykład. Realizację

$$r_1(A) \rightarrow r_2(B) \rightarrow w_1(A) \rightarrow r_2(A) \rightarrow w_2(B) \rightarrow c_2 \rightarrow c_1$$

możemy otrzymać z realizacji

Konfliktowa szeregowalność

Powiemy, że realizacja jest **konfliktowo szeregowalna**, jeżeli jest konfliktowo równoważna pewnej realizacji sekwencyjnej.

Jeżeli możemy otrzymać pewną realizację R z realizacji sekwencyjnej przez zamianę operacji niekonfliktowych, to R jest konfliktowo szeregowalna.

Przykład. Realizację

$$r_1(A) \rightarrow r_2(B) \rightarrow w_1(A) \rightarrow r_2(A) \rightarrow w_2(B) \rightarrow c_2 \rightarrow c_1$$

możemy otrzymać z realizacji

$$r_1(A) \rightarrow w_1(A) \rightarrow r_2(B) \rightarrow r_2(A) \rightarrow w_2(B) \rightarrow c_2 \rightarrow c_1,$$

a zatem również z realizacji

Konfliktowa szeregowalność

Powiemy, że realizacja jest **konfliktowo szeregowalna**, jeżeli jest konfliktowo równoważna pewnej realizacji sekwencyjnej.

Jeżeli możemy otrzymać pewną realizację R z realizacji sekwencyjnej przez zamianę operacji niekonfliktowych, to R jest konfliktowo szeregowalna.

Przykład. Realizację

$$r_1(A) \rightarrow r_2(B) \rightarrow w_1(A) \rightarrow r_2(A) \rightarrow w_2(B) \rightarrow c_2 \rightarrow c_1$$

możemy otrzymać z realizacji

$$r_1(A) \rightarrow w_1(A) \rightarrow r_2(B) \rightarrow r_2(A) \rightarrow w_2(B) \rightarrow c_2 \rightarrow c_1,$$

a zatem również z realizacji

$$r_1(A) \rightarrow w_1(A) \rightarrow c_1 \rightarrow r_2(B) \rightarrow r_2(A) \rightarrow w_2(B) \rightarrow c_2.$$

Oznacza to, że wyjściowa realizacja jest konfliktowo szeregowalna.

Graf zależności

Niech dana będzie pewna realizacja R zbioru transakcji $\mathcal{T} = \{T_1, \dots, T_n\}$. **Graf zależności** (lub **konfliktów**) dla $\mathcal{T}$ jest grafem skierowanym, w którym wierzchołki odpowiadają transakcjom, natomiast krawędzie skierowane od T_i do T_j , gdzie $i \neq j$, istnieje wtedy, gdy w realizacji R występują operacje konfliktowe $o_i \rightarrow o_j$ takie, że o_i należy do transakcji T_i , natomiast o_j do T_j .

Graf zależności

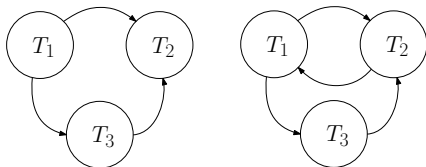
Niech dana będzie pewna realizacja R zbioru transakcji $\mathcal{T} = \{T_1, \dots, T_n\}$. **Graf zależności** (lub **konfliktów**) dla $\mathcal{T}$ jest grafem skierowanym, w którym wierzchołki odpowiadają transakcjom, natomiast krawędzie skierowane od T_i do T_j , gdzie $i \neq j$, istnieje wtedy, gdy w realizacji R występują operacje konfliktowe $o_i \rightarrow o_j$ takie, że o_i należy do transakcji T_i , natomiast o_j do T_j .

Przykład. Dla realizacji

$R_1 : \quad r_1(A) \rightarrow r_3(B) \rightarrow w_1(A) \rightarrow w_2(B) \rightarrow r_3(A) \rightarrow w_2(A)$

$R_2 : \quad r_1(A) \rightarrow r_3(B) \rightarrow w_1(A) \rightarrow w_2(B) \rightarrow r_3(A) \rightarrow r_1(B) \rightarrow w_2(A)$

mamy następujące grafy konfliktów:



Konfliktowa szeregowalność a grafy konfliktów

Twierdzenie

Realizacja jest konfliktowo szeregowalna wtedy i tylko wtedy, gdy odpowiadający jej graf konfliktów nie zawiera cykli.

Konfliktowa szeregowalność a grafy konfliktów

Twierdzenie

Realizacja jest konfliktowo szeregowalna wtedy i tylko wtedy, gdy odpowiadający jej graf konfliktów nie zawiera cykli.

Dowód (przez machanie rękami).

Założmy, że realizacja jest konfliktowo szeregowalna. Istnieje zatem konfliktowo równoważna realizacja sekwencyjna R' . Gdyby graf zawierał cykl, to mielibyśmy dla pewnych dwóch transakcji T_i i T_j dwie pary konfliktowych operacji $(o_{i_1}, o_{j_1}), (o_{i_2}, o_{j_2}) \in T_i \times T_j$ takie, że $o_{i_1} \rightarrow o_{j_1}$ oraz $o_{j_2} \rightarrow o_{i_2}$ w R' , co przeczy konfliktowej szeregowalności.

Założmy, że graf konfliktów dla realizacji nie zawiera cykli. Oznacza to, że możemy transakcje wchodzące w skład realizacji posortować topologicznie. Otrzymany porządek liniowy na transakcjach odpowiada realizacji sekwencyjnej równoważnej wyjściowej realizacji. □

Czy konfliktowa szeregowalność wystarcza?

Konfliktowa szeregowalność gwarantuje poprawność dowolnej realizacji przy założeniu, że realizacje zawierają tylko zatwierdzone transakcje. Co się jednak dzieje w przypadku awarii lub wycofania którejś z transakcji?

Czy konfliktowa szeregowalność wystarcza?

Konfliktowa szeregowalność gwarantuje poprawność dowolnej realizacji przy założeniu, że realizacje zawierają tylko zatwierdzone transakcje. Co się jednak dzieje w przypadku awarii lub wycofania którejś z transakcji?

Przykład. W przypadku realizacji

$$\begin{aligned} r_1(A) \rightarrow w_1(A) \rightarrow r_1(B) \rightarrow r_2(A) \rightarrow w_1(B) \rightarrow r_2(B) \\ \rightarrow c_2 \rightarrow r_1(C) \rightarrow w_1(C) \rightarrow \text{awaria} \end{aligned}$$

pojawia się niebezpieczeństwo brudnego odczytu.

Zatem konfliktowa szeregowalność nie wystarcza do zapewnienia poprawności realizacji: muszą być wprowadzone dodatkowe warunki.